# Virtual Threads

Virtual Threads · Fan-out 10k blocking HTTP checks · Build with Waqar

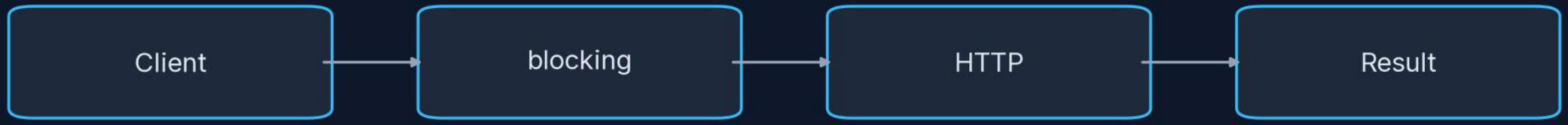
# Today's Hook

- Problem: Fan-out 10k blocking HTTP checks
- Pattern: Virtual Threads — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

# The Problem

- Fan-out 10k blocking HTTP checks
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

# Structure




Virtual Threads: Fan-out 10k blocking HTTP checks

# Real-World Scenario

- Running example: Fan-out 10k blocking HTTP checks
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

# Naive Code

```java
package com.buildwithwaqar.patterns.concurrency.virtualthreads;

import com.buildwithwaqar.patterns.concurrency.virtualthreads.VirtualThreadDemo;
/** Problem: fan-out thousands of blocking health checks without exhausting OS threads. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Virtual Threads problem: massive fan-out ===");
    System.out.println("checks=" + VirtualThreadDemo.fanOut(200));
  }
}
```

# Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

# Solution

```java
package com.buildwithwaqar.patterns.concurrency.virtualthreads;

import com.buildwithwaqar.patterns.concurrency.virtualthreads.VirtualThreadDemo;
/** Problem: fan-out thousands of blocking health checks without exhausting OS threads. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Virtual Threads problem: massive fan-out ===");
    System.out.println("checks=" + VirtualThreadDemo.fanOut(200));
  }
}
```

# Runnable main

```java
package com.buildwithwaqar.patterns.concurrency.virtualthreads;

import com.buildwithwaqar.patterns.concurrency.virtualthreads.VirtualThreadDemo;
/** Problem: fan-out thousands of blocking health checks without exhausting OS threads. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Virtual Threads problem: massive fan-out ===");
    System.out.println("checks=" + VirtualThreadDemo.fanOut(200));
  }
}
```

# Walkthrough Output

```
=== Virtual Threads problem: massive fan-out ===
checks=200
```

# Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

# Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Virtual Threads.

# Related Patterns

- Contrast Virtual Threads with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Problem → naive → Virtual Threads → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.